

# 镜像 Image

## 第 4 讲

# 目标

理解镜像是只读包，不是什么迷你操作系统

- 本质:只读包 · 共享宿主机内核 · 很小
- 记忆:镜像=图纸,容器=盖出来的房子
- 分层:核心概念,省磁盘/带宽/加速构建
- 命令:pull/images/push/rmi 为核心

# 一、镜像的"真身"

一幅只读的图纸，不是一台操作系统

# 镜像 = 能跑应用的整套"环境"，打包成静态文件

-  应用程序本体与依赖
-  最小化 OS 构造(不含内核)
-  元数据: 端口 / 环境变量 / 启动命令

关键词:只读——一旦创建，永不修改

# Alpine 只有 7MB，因为它没带内核

- 没有完整 shell / 没有包管理器 / 没有内核
- 容器与宿主机共享同一个内核
- Windows 镜像则动辄 GB 级

# 镜像是图纸，容器是盖出来的房子

| 维度   | 镜像 Image               | 容器 Container     |
|------|------------------------|------------------|
| 时机   | <b>构建时</b> 的静态模板       | <b>运行时</b> 的可写实例 |
| 可变性  | 只读，不可改                 | 可写，可跑可改可挂卷       |
| 生命周期 | 持久存在                   | 可启停销毁            |
| 关系   | 一个镜像 → 启动 <b>N</b> 个容器 | 各自独立             |

也像：类之于对象、程序之于进程

## 二、镜像的身份与仓库

名字定项目，标签定版本，仓库管分发

# 标签是"铭牌"，latest 不代表最新

- 格式: 镜像名:标签 → `ubuntu:v2`
- 不写标签 → 自动挂 **latest**
- ⚠ latest 只是默认名, ≠ 最新版本
- 同一份数据可挂不同标签(viewport)

`docker tag` 就是给同一镜像 ID 加别名, 不复制数据

# 镜像存在仓库(Registry)里，默认 Docker Hub

| 概念         | 说明                  |
|------------|---------------------|
| Registry   | 存放/分发镜像的服务端         |
| Repository | 按项目组织的镜像集合，如 alpine |
| Docker Hub | 官方公共仓库，最大           |

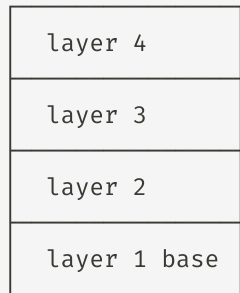
```
[registry/]repository[:tag]  
docker pull python:3.12-slim    # 省略 registry → Docker Hub
```



## 三、镜像的本质:分层

一层一层堆叠，还能跨镜像共享

# 镜像是多层只读层的堆叠



最上层

← Alpine 基础镜像



# 层可以共享：省磁盘 · 省带宽 · 加速构建

Alpine 基础层 — 被镜像 A、B、C 共同使用(只存一份)

A | B | C 各自镀上自己的"专属层"



- 磁盘上只存一份共享层
- 拉新镜像只取未命中的层
- 构建缓存命中不变的部分, 秒级完成

# 容器能写只读镜像，靠的是"写时复制"COW

- 启动时在镜像上叠一层可写容器层
- 读 → 直接读底层只读副本
- 写 → 先把文件**复制**到容器层再改
- 容器删除 → 可写层销毁 → **镜像毫发无损**

# 常用的镜像命令

| 命令                             | 作用                  |
|--------------------------------|---------------------|
| <code>pull</code>              | 拉取镜像(非必需, run 会自动拉) |
| <code>images / ls</code>       | 查看本机镜像              |
| <code>push</code>              | 推送镜像到仓库             |
| <code>rmi</code>               | 删除镜像(有容器引用删不掉)      |
| <code>tag</code>               | 加标签/改名(不复制数据)       |
| <code>history / inspect</code> | 看分层 / 看 JSON 元数据    |
| <code>save / load</code>       | 离线导出 / 导入 tar       |

## 四、实战一:拉取并解剖一个镜像

亲手 pull → run → history → inspect, 看见"分层"

# 拉取让分层"现形"

```
docker pull hello-world      # 分步显示 Pulling fs layer → Pull complete
docker run --rm hello-world  # 打印 Hello from Docker! 后退出
docker pull alpine:3.20      # 每个 Pull complete 对应一层
```



- 每个 Pull complete = 一层, 这就是分层的实感
- --rm = 退出后自动删容器

# history / inspect 把层列给我看

```
docker history alpine:3.20
```

# 一行 CMD(0B) 叠在一行 ADD(7.38MB) 上 → 每行就是一层

```
docker inspect alpine:3.20
```

```
docker inspect alpine:3.20 --format '{{.Architecture}}'
```



- history 直接呈现分层结构
- inspect 返回完整 JSON: ExposedPorts / Env / Architecture

## 五、实战二:把容器变成镜像

docker commit + tag + save + rmi + load, 走通镜像生命周期

# commit: 把容器的可写层定格成镜像

```
docker run -it --name myubuntu ubuntu:22.04 /bin/bash # ① 进容器
apt-get update && apt-get install -y curl vim          # ② 做修改
docker commit myubuntu myubuntu:with-tools            # ③ 定格为镜像
```



- 修改写入容器的可写层, 底层 ubuntu 镜像没变
- commit 不是正式构建方式, 正式项目用 Dockerfile

# tag → save → rmi → load: 一次镜像"备份与恢复"

```
docker tag myubuntu:with-tools jamelouis/myubuntu:v1 # 加别名
docker save -o myubuntu.tar jamelouis/myubuntu:v1    # 导出离线文件
docker rm myubuntu && docker rmi myubuntu:with-tools jamelouis/myubuntu:v1
docker load -i myubuntu.tar                          # 导入恢复
```



同一 IMAGE ID 挂多个名 = tag 加别名的直观体现; 可选 push 到 Hub

# 总结：一张表带走全部要点

| 方面       | 要点                                                        |
|----------|-----------------------------------------------------------|
| 本质       | 只读包 · 不含内核 · 共享宿主内核 → 很小(Alpine ~7MB)                     |
| 镜像 vs 容器 | 构建时模板 vs 运行时实例; 一图启动 N 房                                  |
| 命名       | 镜像名 **: ** 标签; latest ≠ 最新; 仓库默认 Docker Hub               |
| 分层       | 多层只读层堆叠 · 跨镜像共享 · 写时复制保护只读性                               |
| 命令       | pull/images/push/rmi 核心; tag/history/inspect/save/load 按需 |

# 从"有图"到"画图"

下讲: 用 Dockerfile 自动化、可复现地构建镜像

# 谢谢大家

Q & A